

Agentropix-SIFT — Main Architectural Design

Audience: architects, examiners, and reviewers who need a single verified picture of the runtime. **Question this page answers:** *what are the moving parts of Agentropix-SIFT, how do LLM clients reach the deterministic tools, and which guardrails are enforced in code versus by convention?*

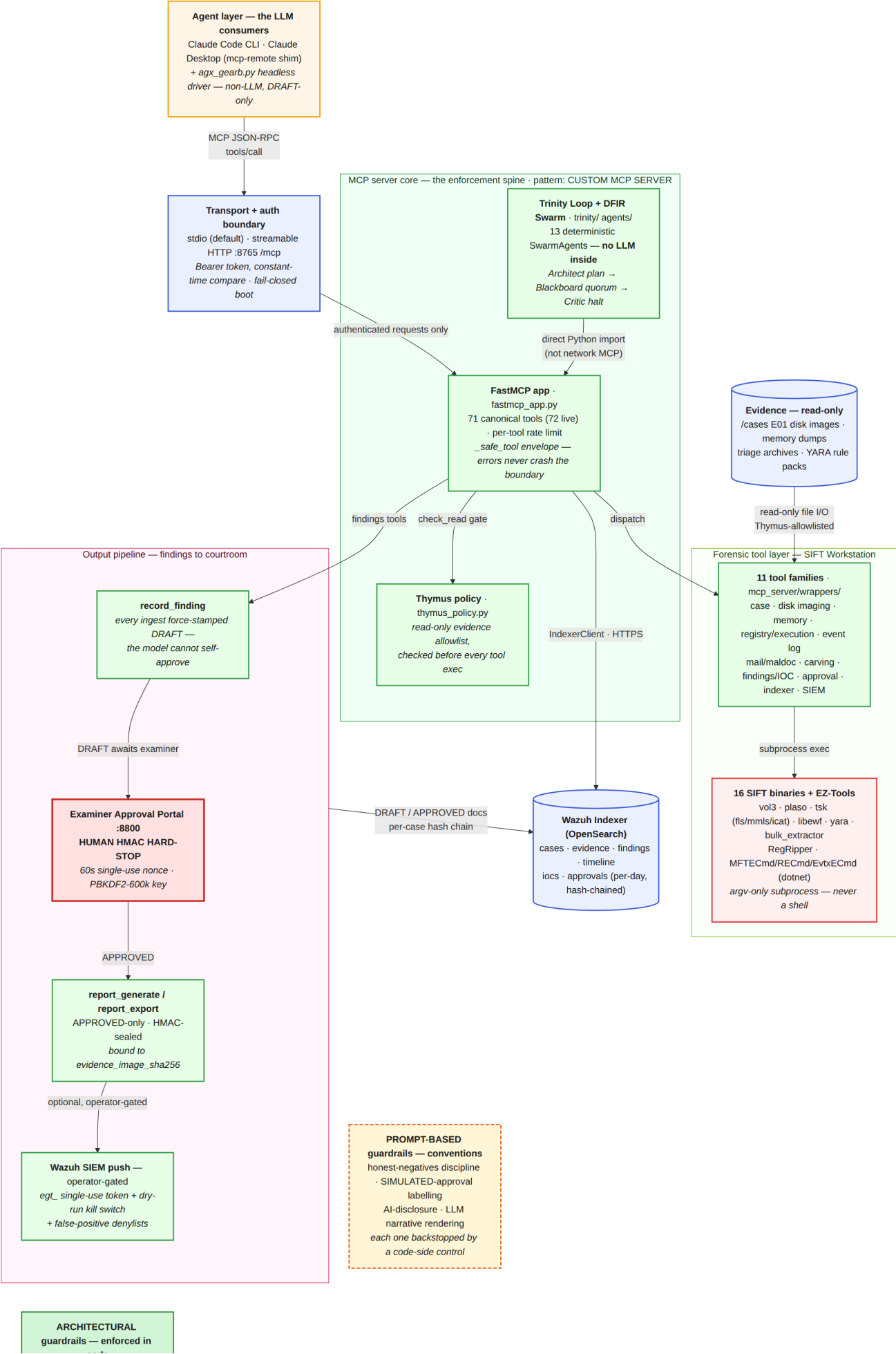
How to read this page

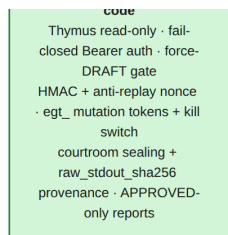
This page presents one validated architecture diagram of the Agentropix-SIFT runtime, derived from the oracle repository (/home/admin2/agentropix-sift , src/ + docs) and reconciled against [canonical-facts.md](#) . Every component, connection, and guardrail shown below is grounded in a named source file (see the [Component-to-source map](#)); claims that did not survive source verification are recorded as honest negatives rather than silently dropped.

Reading order:

1. **The diagram** — top-to-bottom: agent layer → transport/auth boundary → MCP server core → tool families → wrappers → SIFT Workstation binaries, with data sources feeding in, persistence at the side, and the findings → approval → sealed-report → Wazuh output pipeline.
2. [The components in detail](#) — a narrative walk through each diagram layer: what the portal documentation says, then what the source code shows, with every disagreement called out (the source wins).
3. **Two guardrail columns** flank the flow: **ARCHITECTURAL** (green, solid border — enforced in code, the model cannot bypass them) and **PROMPT-BASED** (amber, dashed border — conventions an LLM is instructed to honor). Each guardrail box names the point it protects (e.g. “protects: approval portal”); the full enforcement-file mapping is in the table below.
4. The **red, thick-bordered node** is the Examiner Approval Portal — the human HMAC hard-stop. No code path promotes a DRAFT finding to APPROVED without a valid challenge-response signature from the examiner.

The diagram is committed as a PNG (rendered from Mermaid source via `mmdc`) so it displays for every reader regardless of client-side Mermaid support.





Agentropix-SIFT architecture

 [Open as SVG — full size, zoomable](#) ·  [High-definition PDF \(vector, print-ready\)](#) ·  [Raster PDF](#) · [Mermaid source](#)

The components in detail — documentation first, then source contrast

Each subsection follows the validation discipline used to build the diagram: **what the portal documentation says** (docs/02-architecture, 04-mcp-tools, 05-safety-forensics, 09-integrations, 10-agents, plus the oracle `README / CANONICAL_FACTS.md`) is stated first, then **what the source code under `src/agentropix_sift/` actually shows**, with every disagreement called out. Where the two conflict, the source is authoritative.

1. The agent

Documentation ([client-setup.md](#), [agentic-architecture.md](#), [user-guide.md](#)): the “agent” is not a bundled LLM — it is whatever MCP client the operator attaches. Three consumer shapes are documented: **Claude Code CLI** (recommended; `claude mcp add --transport http` with a Bearer header), **Claude Desktop** (stdio-only GUI, bridged to the HTTP endpoint via an `npx mcp-remote` shim, 1 MB inline-result cap), and a **local stdio launch** via an `mcp.json` `command` entry. The dual-audience model means a non-technical user types a plain-language prompt and the session routes it to a real MCP tool.

Source contrast — confirmed, with two sharpenings:

- The literal `mcp.json` snippet is shipped *inside the module docstring* of the server package itself — the product is explicitly built to be mounted into a general-purpose Claude session.
- There are also **non-LLM agents**. `agx_gearb.py` is a deterministic headless Python driver (one persistent `Mcp-Session-Id` HTTP session, per-step `SUMMARY.json` checkpoints, launched detached); it lives on the workstation **outside the product repo** and by design stages findings as DRAFT only, stopping at the approval gate. Inside the package, the **Trinity Loop + DFIR Swarm** (`trinity/`, `agents/`, `detectors/`) coordinates 13 `SwarmAgent` classes (7 core specialists + 6 ATT&CK detectors) over an asyncio-locked Blackboard — and the source is explicit that this is **LLM-free** (“no LLM coupling — pure async coroutines”). The *only* in-runtime LLM touchpoint in the entire package is the optional Architect reorder pass (`AGENTROPIX_ARCHITECT_LLM_REORDER`, default **off**, fail-open, Claude haiku), whose output is code-validated (see the guardrail table).
- The BMAD/forged personas and OpenClaw crews that appear in the docs are **build-time and documentation-production roles only**; the source contains no runtime trace of them. A reader could misread the docs as implying a runtime multi-agent product — the code says otherwise.

2. The SIFT Workstation tools

Documentation ([tool-list.md](#), [canonical-facts.md](#)): 16 SIFT forensic binaries driven by wrapper modules, surfaced as 71 MCP tools grouped into families.

Source contrast — the concrete inventory:

- The SIFT-16** (verified in the `doctor` preflight dict, `cli.py:176-196`): `vol` (Volatility 3), `log2timeline.py` (Plaso, 6 workers), `fls / icat / mmls` (Sleuth Kit), `ewfinfo` (libewf), `evtx_dump.py` (python-evtx; the Rust `evtx_dump` preferred when present), `yara`, `bulk_extractor`, `rip.pl` (RegRipper), `pf`, `amcache_parser`, `shimcache_parser`, `exiftool`, `foremost`, `hashdeep`.
- Eric Zimmerman tools** run as genuine .NET binaries — `dotnet /opt/ezt/net9/Tool/Tool.dll` — for MFTECmd, RECcmd, LECcmd, JLECcmd, SBECcmd, SQLECmd, and `bstrings` (stdin-piped, W-130).
- Auxiliary binaries** outside the 16: `esedbexport` (SRUM parsing — *libesedb*, not `SrumECmd`), `evtxexport`, `strings`, `rabin2`, `capa`, `xxd`, `sgdisk`, `qemu-img`, `7z / tar`. Maldoc analysis (`olevba / oleid / rtfobj`) is an **in-process Python library** (`oletools`, W-221), not a shelled binary.
- Invocation discipline** (`mcp_server/wrappers/_subprocess.py`): everything runs via `asyncio.create_subprocess_exec` with an `argv` list — **never a shell**. Exactly **7 memory-heavy wrappers** (volatility, plaso, bstrings, jlecmed, sbecmed, sqlecmd,

pdf_extract_text) get an RSS memory cap (max(4096 MB, image_GB×730) , W-162); the rest are timeout-kill only.

- **Documentation corrections found:** env-name overrides (AGENTROPIX_*_TOOL) exist for only **11** tools — vol , fls / mmls , ewfinfo , rip.pl resolve via bare shutil.which , contrary to an “every wrapper” phrasing in earlier drafts. The wrapper directory is mcp_server/wrappers/ with **59 .py files** (~52 wrapper modules + 7 support) — the canonical-facts “~40 files” row is stale, and the top-level wrappers/ package (3 modules) is *not* the tool layer.

3. The MCP server

Documentation ([mcp-server.md](#), [fastmcp-execution.md](#)): a single FastMCP server, two transports, the Thymus boundary, 71 tools.

Source contrast — confirmed with precision added:

- One **FastMCP 3.2.4** app named agentropix-sift (mcp_server/fastmcp_app.py:_build_app), decorator-registered: 67 @app.tool() in fastmcp_app.py + 5 via the Wazuh registrars = **72 live registrations** at HEAD vs the canonical **71** — a persistent off-by-one that the docs themselves resolve by designating health's len(await app.list_tools()) as the live source of truth. (A portal note claiming wazuh_hunt_ioc is “registered twice” is **wrong** — the source shows exactly one registration; the surplus grep hits are docstrings in _safe_tool.py.)
- **Transports:** stdio is the default (app.run() , trust = process UID); streamable HTTP is opt-in on **:8765 at /mcp** , loopback unless explicitly exposed (ADR-017 tailnet-only). Stale /sse strings in the source are comments only.
- **Auth:** BearerTokenMiddleware with secrets.compare_digest (constant-time, SIFT-W-281), injected by wrapping app.http_app() — FastMCP 3.x removed the .app shim, so it is NOT app.run(middleware=[...]) . Boot is **fail-closed**: _build_app() raises without AGENTROPIX_MCP_AUTH_TOKEN unless AGENTROPIX_MCP_DEV_MODE=1 — even a stdio launch refuses to start.
- **The enforcement spine:** the shared tool core server.py (61 mcp_* async functions) carries the in-code statement “The MCP server is the enforcement boundary — Thymus policy runs here, not in the agent.” A per-tool sliding-window _RateLimiter (default 60/min) and **47 ThymusEvidencePolicy.check_read call sites** sit in front of every wrapper execution.
- On the plural “MCP servers”: there is **one** MCP server. The Examiner Approval Portal (:8800) and the Wazuh services are companion HTTP services, not MCP servers.

4. The data sources

Documentation ([persisted-artifacts.md](#), [data-models.md](#)): evidence under /cases/ , YARA rules, case state, indexer persistence.

Source contrast:

- **Evidence (read-only inputs):** disk images (E01/raw/GPT), memory dumps, triage archives, PST/email — readable only under the Thymus-allowlisted prefixes /cases/ , /mnt/ , /media/ , /evidence/ , /tmp/agentropix-sift-*. check_write() on evidence **unconditionally rejects** — the agent literally has no tool to write evidence.
- **YARA rule packs:** bundled under detectors/yara_rules/ (e.g. cobalt_strike_loader.yar) plus the system rule directories.
- **Case state:** a one-line active-case pointer at ~/.agentropix/active_case (AGENTROPIX_ACTIVE_CASE_DIR overridable) resolves the default case_id for case-scoped tools.
- **Persistence** (outputs that later become inputs): the **Wazuh Indexer** (OpenSearch fork, :9200; IndexerClient over httpx HTTPS + Basic Auth + tenacity retry) holding agentropix-cases (single doc, _id=case_id) , -evidence-*, -findings-*, -timeline-*, -iocs-*, and per-day -approvals-YYYY.MM.DD indices with shipped index templates and ISM policies; the egt_token registry in SQLite (~/.agentropix/evidence-gate.sqlite , mode 0600, tokens stored as SHA-256 hashes, atomic verify-and-spend, 7-day TTL cap); and the audit artifacts (Thymus JSONL + in-memory ring, /var/log/agentropix/http_audit.log , batch-push audit JSONL + 180-day dedup cache).
- **Documentation correction:** the Hippocampus “memory” module is opt-in (default OFF) and holds **in-memory-only** reasoning traces for Trinity recall — it is *not* a persistence indexer, despite how the docs could read.

5. The output pipeline

Documentation ([human-in-the-loop.md](#), [audit-courtroom.md](#), ADR-016/022/024) and source agree on the shape; the source pins the mechanics:

1. **record_finding / wazuh_index_findings** — every ingest is **force-stamped DRAFT** (the W-286 gate strips any caller-supplied approval.*), idempotent on (case_id, finding_id) ; live writes additionally require a single-use egt_token . Writes go to agentropix-findings-*, *not* the Wazuh alerts index.
2. **Examiner Approval Portal** (Starlette, :8800, W-288) — the human hard-stop: POST /challenge issues a 60-second single-use nonce; POST /approve requires an HMAC-SHA256 signature with a PBKDF2-600k-derived key from the examiner's password, plus the BUG-001 precondition gate (the target must exist and currently hold the asserted from_status). **Source correction:** the per-case prev_approval_hash chain (Phase 2) **has shipped** — approval_sidecar/writer.py:141-328 backfills it before every bulk write and verify_approval_chain walks it; portal prose still calling it “deferred” is stale.

- 3. `report_generate` — 6 query profiles (full/executive/timeline/ioc/findings/status); reconciles **APPROVED-only** findings and warns when 0 APPROVED but DRAFTs exist. `report_export` (ADR-024) projects the result into 3 audience tiers × md/html/pdf — presentation-only (“adds no new evidence”); the pdf path is capability-gated and never auto-installs.
- 4. **Courtroom sealing** — HMAC-SHA256 with a per-run 32-byte session key (mode 0600); the audit-log seal is cross-bound into the report *before* the report seal (ADR-022), and `evidence_image_sha256` binds the report to the image bytes (1 MiB chunks, 50 GB cap with an honest skip). `verify_seal.py` / `provenance/validate.py` are exit-code hard gates.
- 5. **Optional SIEM egress** — `wazuh_publish_iocs` to the Wazuh **Manager** API (:55000, HTTPS+JWT — a separate auth chain from the Indexer’s Basic Auth) pushing `agentropix_*` CDB lists + rules with one coalesced restart; gated by `egt_` token, false-positive denylists, and the `WAZUH_DRY_RUN_ONLY` kill switch (checked *before* token verification). **Source correction:** the routing table (`wazuh/orchestrator.py`) defines **8** CDB list kinds, not the “6” some pages state. A separate `BatchPushOrchestrator` maps findings→alerts (`wazuh-alerts-dfir-STAGING-*` on dry-run, live via the Manager) with a 180-day dedup cache; shipped dashboards (`agentropix-findings.ndjson`) provide the Findings/Timeline views.

Architectural pattern

Verdict: Custom MCP Server.

Agentropix-SIFT is a purpose-built FastMCP 3.2.4 server (`FastMCP("agentropix-sift")`), entry point `agentropix-sift-mcp`) exposing 71 canonical / 72 live-registered forensic tools to general-purpose MCP clients. Claude Desktop and Claude Code attach via a literal `mcp.json` snippet shipped in the module docstring, over stdio (the default transport) or Bearer-protected streamable HTTP at `http://TAILNET-HOST:8765/mcp` . The server itself is the enforcement boundary: the Thymus read-only path policy, the rate limiter, the W-286 DRAFT gate, and the fail-closed Bearer auth all run server-side — *the agent literally has no tool to write evidence* — and the package ships no LLM client of its own in the MCP path. It is **not** a Direct Agent Extension (no Claude-internal plugin code) and **not** an Alternative Agentic IDE (no editing environment).

The verdict “*LLM at the edge, determinism inside*” is verified end-to-end: every fact originates from a named deterministic tool wrapping one of 16 SIFT Workstation binaries via no-shell subprocess execution, and the only in-runtime LLM call in the entire package is the optional, default-off, fail-open Architect reorder pass (`AGENTROPIX_ARCHITECT_LLM_REORDER`).

Hybrid note — secondary elements that must not be mislabeled. The same package ships an in-process Trinity-Loop/Blackboard swarm: 13 SwarmAgent classes (7 core specialists + 6 ATT&CK detectors), explicitly LLM-free, invoked by the local `agentropix-sift run` CLI and calling the tool core via **direct Python import, not network MCP** — a deterministic multi-agent runtime, not an LLM Multi-Agent Framework. The `agx_gearb.py` headless driver (non-LLM, workstation-side, outside the repo, DRAFT-only — it stops at the approval gate) and the BMAD/OpenClaw forge crews (build-time and documentation-production roles only) sit **outside** the product runtime diagram.

Guardrails: prompt-based vs architectural

The distinction matters in court: an **ARCHITECTURAL** guardrail is enforced by code the model cannot reach around; a **PROMPT-BASED** guardrail is an instruction the model is expected — but not forced — to honor. Agentropix-SIFT pairs every load-bearing prompt-based control with a code-side backstop.

Name	Type	Enforced by	What it does
Thymus read-only evidence policy	ARCHITECTURAL	<code>mcp_server/thymus_policy.py</code> + <code>server.py</code> (47 <code>check_read</code> sites)	Allowlist prefixes + forbidden patterns (<code>..</code> , <code>~</code> , <code>/dev</code> , <code>/proc</code> , <code>/sys</code>), URL-decode/symlink/PATH_MAX defenses; <code>check_write()</code> unconditionally rejects — read-only scoped to EVIDENCE (derived output still written to allowlisted <code>out_dirs</code>).
Fail-closed Bearer auth (HTTP)	ARCHITECTURAL	<code>mcp_server/fastmcp_app.py</code> :173-306	<code>AGENTROPIX_MCP_AUTH_TOKEN</code> required on every <code>/mcp</code> request, constant-time compare (SIFT-W-281); <code>_build_app()</code> refuses to start without a token unless <code>AGENTROPIX_MCP_DEV_MODE=1</code> (even stdio launches).
HTTP request audit log	ARCHITECTURAL	<code>fastmcp_app.py</code> :310-335	JSON-lines audit of every <code>/mcp</code> request to <code>/var/log/agentropix/http_audit.log</code>

Name	Type	Enforced by	What it does
			(dir env-overridable); the token is never logged, only <code>sha256[:16]</code> .
Per-tool rate limiter	ARCHITECTURAL	<code>mcp_server/server.py:194-254</code>	Sliding-window per-tool limit (default 60/min, per-tool env override) under <code>threading.Lock</code> for the HTTP worker pool.
<code>_safe_tool_error</code> envelope (WZ-021)	ARCHITECTURAL	<code>mcp_server/wrappers/_safe_tool.py</code>	Every tool exception becomes a flat <code>{error, details}</code> envelope — failures never crash the FastMCP boundary (KeyboardInterrupt/SystemExit/CancelledError propagate).
W-286 DRAFT gate	ARCHITECTURAL	<code>mcp_server/wrappers/wazuh_tools.py:40-127</code> (+ <code>case_records.py</code> , <code>server.py:1273</code>)	<code>_apply_draft_gate</code> strips caller-supplied <code>approval.*</code> and forces <code>status=DRAFT</code> on every indexed finding — the LLM cannot self-approve via any write surface.
HMAC approval sidecar (human hard-stop)	ARCHITECTURAL	<code>approval_sidecar/app.py</code> + <code>auth.py</code> + <code>nonce.py</code>	DRAFT→APPROVED requires challenge-response: 60 s single-use nonce + HMAC-SHA256 with a PBKDF2(600k, env-tunable)-derived key from the examiner password; no code path writes an approval without a valid signature.
Nonce anti-replay store	ARCHITECTURAL	<code>approval_sidecar/nonce.py:49-113</code>	Single-use, TTL-bound, (examiner, target)-bound nonces; consumed even on failure so replay and cross-target approval are rejected in code.
Approval precondition gate (BUG-001)	ARCHITECTURAL	<code>approval_sidecar/app.py:225-260, 379-384</code>	The target must exist in the case AND currently hold the asserted <code>from_status</code> before an approval is written (409 on phantom/double/stale approvals).
Approval hash chain (Phase 2 live)	ARCHITECTURAL	<code>approval_sidecar/writer.py:141-328</code>	Per-CASE <code>prev_approval_hash</code> backfilled before every bulk write (BUG-002 fix); <code>verify_approval_chain</code> walks the chain; the deterministic <code>approval_id</code> includes the nonce (but is NOT wired as the OpenSearch <code>_id</code> — <code>bulk_index</code> auto-generates).
<code>dry_run=True</code> defaults + <code>egt_</code> mutation tokens	ARCHITECTURAL	<code>wazuh/evidence_gate.py</code> + <code>evidence_gate/registry.py</code>	Live mutation needs a single-use, scoped, TTL-capped (7 d) <code>egt_</code> token, hash-stored and atomically verify+spent in SQLite, fail-closed if the verifier is unimportable. NOT universal: <code>record_timeline_event</code> has no <code>dry_run/token</code> ; the <code>AGENTROPIX_EVIDENCE_GATE_STEP1_STUB</code> hatch degrades verification to format-only.
<code>WAZUH_DRY_RUN_ONLY</code> kill switch	ARCHITECTURAL	<code>wazuh/config.py:236</code> + <code>wazuh_tools.py:200-208, 786-794</code>	Env-level boolean (default True) checked BEFORE token verification — rejects any live write regardless of token; the model cannot toggle it.
FP denylists + RFC1918 gating	ARCHITECTURAL	<code>wazuh/denylists.py</code>	Regex/provenance-aware false-positive gate before any IOC reaches Wazuh; internal IPs gated by <code>accept_internal_ips</code> + <code>WAZUH_OPERATOR_TRUSTED_CIDRS</code> .
Courtroom HMAC sealing + cross-binding	ARCHITECTURAL	<code>courtroom.py</code> (ADR-016/ADR-022)	HMAC-SHA256 with a per-run 32-byte session key (0600); audit-log seal embedded into the report before the report seal; <code>evidence_image_sha256</code> binds the report to image bytes (honest skip over the 50 GB cap).
Seal re-verification CLIs (hard gates)	ARCHITECTURAL	<code>audit/verify_seal.py</code> + <code>provenance/validate.py</code>	Row-by-row HMAC re-verification; exits non-zero iff forged/malformed (<code>schema_failed</code> for provenance) — used as run preflight/teardown gates.

Name	Type	Enforced by	What it does
<code>raw_stdout_sha256</code> provenance envelope	ARCHITECTURAL	<code>wrappers/tsk.py</code> , <code>evt.py</code> , <code>schema/pdf_extract_text.py</code> et al.	Wrapper response models carry the SHA-256 of the actual subprocess stdout, grounding every parsed result to the deterministic tool's raw bytes.
<code>report_generate</code> APPROVED-only filter	ARCHITECTURAL	<code>mcp_server/wrappers/case_records.py:1056-1180</code>	Findings/timeline/executive/full profiles include only APPROVED findings (code-side query filter); explicit warning when 0 APPROVED but DRAFTs exist.
<code>delete_finding</code> DRAFT-only	ARCHITECTURAL	<code>case_records.py:336-454</code>	Refuses in code to delete APPROVED/REJECTED/REVOKED findings; the live delete query is additionally scoped to <code>approval.status=DRAFT</code> ; <code>dry_run</code> default.
Deterministic Critic halt + completion proofs	ARCHITECTURAL	<code>trinity/critic.py</code> + <code>orchestrator.py</code>	The Trinity stop decision is a pure function of Blackboard state (findings fingerprint, 0.85 threshold, coverage guard); completion-promise tokens issued only when an agent completed AND published ≥ 1 finding — no LLM in the loop.
Architect LLM-reorder output validation	ARCHITECTURAL	<code>trinity/architect.py:~407-424</code>	Code-side backstop on the one prompt-based runtime feature: the reordered agent set must exactly equal the deterministic set (duplicates rejected, falls through to deterministic order).
Credential redaction (fail-closed)	ARCHITECTURAL	<code>security/redact.py</code>	Credential patterns → <code>[REDACTED-hmac-tag]</code> ; any uncaught exception raises <code>RedactionError</code> so aggregation aborts rather than emitting unredacted output.
Canonical-facts CI drift gate	ARCHITECTURAL	<code>scripts/check_canonical_facts.py</code> (oracle CI)	The build fails on backward/forward numeric drift in tracked docs — documentation numbers are machine-checked (the portal-side “never contradict” rule is the prompt-based companion).
Architect LLM reorder system prompt	PROMPT-BASED	<code>trinity/architect.py:~64-79</code>	“Output JSON order, do not add or remove agents” instruction to Claude haiku — violable by the model; harm prevented by the code-side set validation above.
SIMULATED-examiner-approval labeling	PROMPT-BASED	<code>docu_agentro/CLAUDE.md:116-118</code>	Recording convention: demo auto-approvals must be labelled “SIMULATED examiner approval (demo only)” — no code enforces the label.
Honest-negatives report discipline	PROMPT-BASED	<code>docu_agentro/CLAUDE.md</code> (case-reports section)	Refuted hypotheses stay in reports as honest negatives; synthesis grounded strictly in <code>confirmed-findings.json</code> — an instruction to drafting/critic agents, not code.
AI-disclosure transparency	PROMPT-BASED	docs/05-safety-forensics/ai-disclosure.md	Documents the three honest sources of non-determinism and AI-vs-deterministic-tool attribution — a transparency convention.
LLM narrative rendering of reports	PROMPT-BASED	<code>case_records.py:1071-1073</code> (docstring)	<code>report_generate</code> returns grounded APPROVED-only sections; the narrative prose layer is rendered by the calling LLM and relies on it honoring those sections.

Known gaps (honest negatives)

Recorded so the guardrail story is not overstated:

- `record_timeline_event` mutates the timeline index with **no** `dry_run` /token gate (rate-limit only; rows are still DRAFT-stamped).
- The `AGENTROPIX_EVIDENCE_GATE_STEP1_STUB` env hatch degrades `egt_` token verification to format-only.

- Thymus on-disk JSONL audit is conditional on `AGENTROPIX_AUDIT_LOG` being set (the in-memory ring always runs).
- The deterministic `approval_id` -as-OpenSearch- `_id` is design intent only — live `bulk_index` auto-generates `_id`, so retry idempotency via deterministic `_id` is not wired.

Component-to-source map

Every box in the diagram traces to oracle source under `/home/admin2/agentropix-sift/src/agentropix_sift/` (paths relative to that package unless noted).

Diagram component	Oracle source	Notes
FastMCP app, stdio + HTTP transports, fail-closed boot	<code>mcp_server/fastmcp_app.py</code> (<code>_build_app</code>)	FastMCP 3.2.4; 67 <code>@app.tool()</code> registrations here + 5 via Wazuh registrars = 72 live; the <code>health</code> tool's <code>len(await app.list_tools())</code> is the designated live tool-count source of truth.
BearerTokenMiddleware + HTTP audit	<code>mcp_server/fastmcp_app.py:173-306, 310-335</code>	Injected by wrapping <code>app.http_app()</code> — FastMCP 3.x removed the <code>.app</code> shim; NOT <code>app.run(middleware=[...])</code> .
Shared tool core (61 <code>mcp_*</code> async functions)	<code>mcp_server/server.py</code>	“The MCP server is the enforcement boundary — Thymus policy runs here, not in the agent.”
<code>_RateLimiter</code>	<code>mcp_server/server.py:194-254</code>	<code>AGENTROPIX_RATE_LIMIT</code> default 60 + per-tool override.
ThymusEvidencePolicy	<code>mcp_server/thymus_policy.py</code>	47 <code>check_read</code> call sites in <code>server.py</code> , including <code>out_dir</code> checks.
Wrapper layer (~52 modules, 59 <code>.py</code> files)	<code>mcp_server/wrappers/</code>	NOT the top-level <code>wrappers/</code> dir (3 standalone modules); the canonical-facts “~40 files” row is stale.
Subprocess discipline	<code>mcp_server/wrappers/_subprocess.py</code>	<code>asyncio.create_subprocess_exec</code> (argv list, never shell); <code>run_with_memory_limit</code> (RSS cap <code>max(4096 MB, GB×730)</code> , W-162) on exactly 7 memory-heavy wrappers; env-name <code>AGENTROPIX_*_TOOL</code> overrides exist for 11 tools, the rest resolve via bare <code>shutil.which</code> .
Error envelope	<code>mcp_server/wrappers/_safe_tool.py</code>	WZ-021.
Wazuh tool registrars	<code>mcp_server/wrappers/wazuh_tools.py</code>	<code>register_wazuh_tools</code> (4) + <code>register_wazuh_intel_tools</code> (1), try/except graceful degradation; <code>wazuh_hunt_ioc</code> is registered exactly once.
E01 FUSE lifecycle	<code>imaging/ewf_lifecycle.py</code>	Synchronous sudo'd <code>subprocess.run</code> (<code>ewfmount</code> → <code>ntfs-3g</code>); tool names validated against <code>sudo-argv[0]</code> injection.
ATT&CK detectors + YARA packs	<code>detectors/</code> + <code>detectors/yara_rules/</code>	6 deterministic detector SwarmAgents.
Trinity Loop (Architect/Critic/orchestrator)	<code>trinity/architect.py</code> , <code>trinity/critic.py</code> , <code>trinity/orchestrator.py</code>	<code>run_triage</code> is a typer CLI path, NOT an MCP tool; the Swarm calls the tool core by direct Python import.
Findings/case/report tools	<code>mcp_server/wrappers/case_records.py</code>	DRAFT gate, APPROVED-only reports, DRAFT-only delete.
Approval sidecar (port 8800)	<code>approval_sidecar/app.py</code> , <code>auth.py</code> , <code>nonce.py</code> , <code>writer.py</code>	Starlette; W-288; Phase-2 hash chain lives in <code>writer.py:141-328</code> .
Evidence-gate tokens	<code>wazuh/evidence_gate.py</code> + <code>evidence_gate/registry.py</code>	SQLite registry at <code>~/.agentropix/evidence-gate.sqlite</code> (0600).

Diagram component	Oracle source	Notes
Wazuh publishing + kill switch + denylists	wazuh/config.py , wazuh/denylists.py , mcp_server/wrappers/wazuh_tools.py	Manager API on https://TAILNET-HOST:55000 (JWT); Indexer on https://TAILNET-HOST:9200 (Basic Auth) — two separate auth chains.
Courtroom sealing + verifiers	courtroom.py , audit/verify_seal.py , provenance/validate.py	ADR-016 / ADR-022.
Credential redaction	security/redact.py	Fail-closed (RedactionError).
CI drift gate	scripts/check_canonical_facts.py	Oracle CI.

Key canonical facts

Governing numeric authority: [docs/08-reference/canonical-facts.md](#) .

- **71 MCP tools** (canonical). The live @app.tool surface at HEAD is **72** (67 in fastmcp_app.py + 4 wazuh_tools + 1 wazuh_intel , no duplicates); the canonical lineage carries a persistent off-by-one and the health tool’s live len(list_tools()) is the designated source of truth — an explicit reconciliation, not a contradiction.
- **16 SIFT forensic binaries** driven by the wrappers: vol , log2timeline.py , fls , icat , mmls , ewfinfo , evtx_dump.py , yara , bulk_extractor , rip.pl , pf , amcache_parser , shimcache_parser , exiftool , foremost , hashdeep — plus Eric Zimmerman tools via dotnet and auxiliary binaries outside the 16.
- **4464 tests** (canonical).
- **72/72 (100%) disk recall** and **108/118 (91.5%) memory recall** (canonical).
- **Python 3.12+** (pyproject requires-python >=3.12); **FastMCP 3.2.4** (an earlier portal “2.x pin” note is stale; canonical is 3.2.4).
- **Transports:** stdio (default) and streamable HTTP on port 8765 at /mcp (http://TAILNET-HOST:8765/mcp), Bearer-protected, fail-closed boot, ADR-017 tailnet-only exposure.
- **11 tool families:** 4 + 10 + 7 + 16 + 6 + 4 + 6 + 7 + 2 + 5 + 5.
- **13 SwarmAgent classes** (7 core specialists + 6 ATT&CK detectors), deterministic and LLM-free; the only in-runtime LLM call is the optional fail-open Architect reorder (default OFF).
- **Approval crypto:** PBKDF2 600k iterations, 60 s single-use nonce, HMAC-SHA256; finding lifecycle DRAFT → APPROVED → REJECTED → REVOKED, every ingest force-stamped DRAFT (W-286).
- **8 routed agentropix * CDB list kinds** (6 self-tested, 5 confirmed live — a bare “6” understates the routing surface).
- **Subprocess discipline:** argv-only, never shell; RSS cap max(4096 MB, image_GB×730) on the 7 memory-heavy wrappers, timeout-kill on the rest.